

1. Что такое Spring Boot и для чего он используется?	3
2. Какие основные компоненты Spring Boot и как они взаимодействуют между собой?	3
3. Что такое Spring Boot Starter и как он упрощает конфигурацию приложения?	3
1. Как работает аннотация @SpringBootApplication?	7
2. Какие шаги необходимо выполнить, чтобы подключить базу данных к приложению на Spring Boot?	7
3. Как использовать Spring Boot Actuator для мониторинга и управления вашим приложением?	7
1. Расскажите, как написать свой стартер?	12
2. Как работает процесс автоматической конфигурации в Spring Boot и какие основные механизмы участвуют в этом процессе?	12
3. Почему можно не указывать версии зависимостей в файле сборки при подключении большинства библиотек?	12
1. Что такое Spring Security и для чего он используется?	17
2. Как включить Spring Security в вашем Spring Boot приложении?	17
3. Какие основные компоненты архитектуры Spring Security вы знаете?	17
1. Как настроить аутентификацию пользователя с использованием базы данных в Spring Security?	22
2. Как реализовать JWT аутентификацию в Spring Security?	22
3. Как можно ограничить доступ к определенным URL в вашем приложении с использованием Spring Security?	22
1. Как настроить многомерную аутентификацию (Multi-Factor Authentication, MFA) с использованием Spring Security?	27
2. Как использовать Spring Security для защиты REST API с помощью OAuth2?	27
3. Как настроить собственный фильтр безопасности в Spring Security и в каких случаях это может быть необходимо?	27

1. Что такое Spring Cache и зачем он используется?.....	32
2. Какие аннотации предоставляет Spring Cache из коробки?.....	32
3. В каких случаях имеет смысл использовать кэширование в приложении?	32
1. Как включить и настроить Spring Cache в Spring Boot-приложении?	36
2. Как работает аннотация @Cacheable, и как определить ключ кэша?	36
3. Чем отличается @CachePut от @Cacheable и @CacheEvict?.....	36
1. Как настроить использование внешних систем кэширования, таких как Redis или Caffeine, в Spring Cache?	40
2. Какие существуют подходы к инвалидации кэша в распределённой среде?	40
3. Какие есть проблемы многопоточности при использовании кэша, и как их избежать (например, cache stampede)?	40
4. Как разделить данные между разными серверами?(consistent hashing) ...	40
5. Что такое "прогревание кэша"?.....	40
1. Что такое Spring Cloud и для чего он используется?	47
2. Какие основные проблемы решает Spring Cloud?	47
3. Перечислите несколько проектов, входящих в состав Spring Cloud.....	47
1. Как Spring Cloud упрощает работу с конфигурационными файлами в микросервисной архитектуре?.....	51
2. Чем отличается Spring Cloud Netflix от Spring Cloud?	51
3. Как в Spring Cloud реализована балансировка нагрузки?	51
1. Опишите процесс настройки централизованного логирования в Spring Cloud.	54
2. Каким образом Spring Cloud Stream обрабатывает сообщения в микросервисной архитектуре?.....	54
3. Как в Spring Cloud реализована паттерн цепочки обязанностей (Chain of Responsibility) для обработки запросов?.....	54

1. Что такое Spring Boot и для чего он используется?
 2. Какие основные компоненты Spring Boot и как они взаимодействуют между собой?
 3. Что такое Spring Boot Starter и как он упрощает конфигурацию приложения?
-

1. Что такое Spring Boot и для чего он используется?

Spring Boot — это надстройка над Spring Framework, предназначенная для **быстрого создания и запуска production-ready приложений** с минимальной ручной конфигурацией.

Ключевая идея

Convention over Configuration — разумные настройки по умолчанию вместо бесконечного XML и @Bean.

Для чего используется Spring Boot:

- быстрое создание **микросервисов**
- разработка **REST API**
- создание **web-приложений**
- запуск приложений как **самодостаточных (fat jar)** без внешнего сервера

Что он убирает:

- ручную настройку сервлет-контейнера (Tomcat/Jetty)
- сложную конфигурацию Spring
- громоздкие XML



Важно на собеседовании:

Spring Boot не заменяет Spring, а упрощает его использование.

2. Основные компоненты Spring Boot и их взаимодействие

1 Auto Configuration

Автоконфигурация — сердце Spring Boot.

- Автоматически настраивает бины на основе:
 - classpath (какие зависимости есть)
 - application.yml / application.properties
 - аннотаций (@ConditionalOn...)

Пример:

- есть `spring-boot-starter-web` → автоматически настраивается:
 - `DispatcherServlet`
 - `Tomcat`
 - `Jackson`
 - MVC инфраструктура
-

2 Starter dependencies

Starters — наборы зависимостей под конкретную задачу.

Пример:

```
spring-boot-starter-web
spring-boot-starter-data-jpa
spring-boot-starter-security
```

Они:

- подбирают совместимые версии библиотек
 - избавляют от `dependency hell`
-

3 Embedded Server

Встроенный сервер приложений:

- `Tomcat` (по умолчанию)
- `Jetty`
- `Undertow`

Позволяет:

```
java -jar app.jar
```

Без внешнего сервера.

4 SpringApplication

Класс запуска приложения:

```
@SpringBootApplication
public class App {
    public static void main(String[] args) {
        SpringApplication.run(App.class, args);
    }
}
```

Что делает:

- поднимает ApplicationContext
 - запускает embedded server
 - применяет auto-configuration
-

5 Externalized Configuration

Внешняя конфигурация:

- application.yml
- application.properties
- переменные окружения
- system properties

Поддерживает профили:

```
application-dev.yml  
application-prod.yml
```

Как всё взаимодействует (кратко):

1. `SpringApplication.run()` запускает контекст
 2. Auto Configuration анализирует classpath
 3. Starter'ы подключают нужные зависимости
 4. Embedded server стартует
 5. Приложение готово к работе
-

3. Что такое Spring Boot Starter и как он упрощает конфигурацию?

Что такое Starter

Spring Boot Starter — это **pom-модуль**, который:

- не содержит кода
- содержит **набор зависимостей под конкретную функциональность**

Пример:

```
implementation 'org.springframework.boot:spring-boot-starter-web'
```

Что входит в `spring-boot-starter-web`:

- Spring MVC
- Tomcat
- Jackson
- Validation
- Logging

Вместо ручного подключения 10+ зависимостей.

Как starter упрощает жизнь:

✗ Без starter:

- сам подбираешь версии
- настраиваешь совместимость
- ловишь конфликты

✓ Со starter:

- одна зависимость
 - проверенные версии
 - автоматическая конфигурация
-

Важный момент для собеседования

Starter не делает магию сам по себе —
магию делает **Auto Configuration**, а starter лишь даёт зависимости для её срабатывания.

1. Как работает аннотация `@SpringBootApplication`?
 2. Какие шаги необходимо выполнить, чтобы подключить базу данных к приложению на Spring Boot?
 3. Как использовать Spring Boot Actuator для мониторинга и управления вашим приложением?
-

1. Как работает аннотация `@SpringBootApplication`?

`@SpringBootApplication` — это **составная (meta)-аннотация**, которая объединяет три ключевые аннотации Spring.

```
@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication.run(Application.class, args);
    }
}
```

Из чего она состоит

```
@SpringBootApplication
@EnableAutoConfiguration
@ComponentScan
```

1 `@SpringBootApplication`

- Специализация `@Configuration`
- Обозначает класс как источник bean-дефиниций

👉 По сути: “это основной конфигурационный класс приложения”

2 `@EnableAutoConfiguration`

- Включает механизм автоконфигурации
- Spring анализирует:
 - classpath
 - properties
 - окружение
- Подключает конфигурации через:
- META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports

👉 Важно: автоконфигурация работает через `@ConditionalOnClass`, `@ConditionalOnMissingBean` и т.п.

3 @ComponentScan

- Сканирует компоненты начиная с пакета, где лежит Application
- Находит:
 - @Component
 - @Service
 - @Repository
 - @Controller

⚠ Частая ошибка: класть Application не в корневой пакет.

Итог (кратко для собеседования)

@SpringBootApplication запускает сканирование компонентов, включает автоконфигурацию и определяет основной конфигурационный класс приложения.

2. Шаги для подключения базы данных в Spring Boot

Разберём **правильный порядок** — именно его ждут на собеседовании.

Шаг 1 Добавить зависимости

JPA + драйвер БД

```
implementation 'org.springframework.boot:spring-boot-starter-data-jpa'
runtimeOnly 'org.postgresql:postgresql'
```

(или MySQL / Oracle / H2)

Шаг 2 Настроить application.yml

```
spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/app_db
    username: app
    password: secret
    driver-class-name: org.postgresql.Driver

  jpa:
    hibernate:
      ddl-auto: validate
    show-sql: true
```

Шаг 3 Создать Entity

```
@Entity
@Table(name = "users")
public class User {

    @Id
    @GeneratedValue
    private Long id;

    private String name;
}
```

Шаг 4 Создать Repository

```
public interface UserRepository
    extends JpaRepository<User, Long> {
}
```

Шаг 5 (Опционально) Профили

```
spring:
  profiles:
    active: dev
# application-dev.yml
spring:
  datasource:
    url: jdbc:h2:mem:test
```

Что делает Spring Boot автоматически

- создаёт DataSource
- настраивает EntityManagerFactory
- подключает TransactionManager

👉 Всё это — **auto-configuration**.

3. Как использовать Spring Boot Actuator для мониторинга и управления?

Spring Boot Actuator — это набор **production-ready** эндпоинтов для:

- мониторинга
 - метрик
 - health-check
 - управления приложением
-

Шаг 1 Подключить зависимость

```
implementation 'org.springframework.boot:spring-boot-starter-actuator'
```

Шаг 2 Включить эндпоинты

```
management:
  endpoints:
    web:
      exposure:
        include: health,info,metrics,env
```

Основные эндпоинты

Endpoint	Назначение
/actuator/health	состояние приложения
/actuator/info	информация о приложении
/actuator/metrics	метрики
/actuator/env	конфигурация
/actuator/beans	список бинов

Health-check пример

```
{
  "status": "UP",
  "components": {
    "db": {
      "status": "UP"
    }
  }
}
```

Используется:

- Kubernetes
 - Load Balancer
 - Monitoring systems
-

Метрики

Actuator использует **Micrometer**:

- Prometheus
- Grafana
- Datadog

```
management:
  metrics:
    export:
      prometheus:
        enabled: true
```

Эндпоинт:

/actuator/prometheus

Безопасность (ВАЖНО)

Actuator эндпоинты **нельзя** открывать все в проде.

Обычно:

- health — публичный
 - остальные — за Spring Security
-

Короткие ответы (если интервьюер торопит)

@SpringBootApplication

Это meta-аннотация, включающая автоконфигурацию, component scan и конфигурационный класс.

Подключение БД

Добавить starter + драйвер, настроить datasource, создать entity и repository — остальное делает Spring Boot.

Actuator

Предоставляет production-ready эндпоинты для мониторинга, health-check и метрик.

1. Расскажите, как написать свой стартер?
 2. Как работает процесс автоматической конфигурации в Spring Boot и какие основные механизмы участвуют в этом процессе?
 3. Почему можно не указывать версии зависимостей в файле сборки при подключении большинства библиотек?
-

1. Как написать свой Spring Boot Starter?

Собственный starter обычно состоит из двух модулей:

```
my-starter
├── my-starter-autoconfigure
└── my-starter
```

1 Модуль *-autoconfigure

Здесь находится вся логика автоконфигурации.

1.1. Класс автоконфигурации

```
@AutoConfiguration
@ConditionalOnClass(MyClient.class)
@EnableConfigurationProperties(MyProperties.class)
public class MyAutoConfiguration {

    @Bean
    @ConditionalOnMissingBean
    public MyClient myClient(MyProperties props) {
        return new MyClient(props.getUrl(), props.getTimeout());
    }
}
```

Ключевые аннотации:

- @AutoConfiguration (Spring Boot 3+)
 - @ConditionalOnClass
 - @ConditionalOnMissingBean
 - @EnableConfigurationProperties
-

1.2. Properties-класс

```
@ConfigurationProperties(prefix = "my.client")
public class MyProperties {

    private String url;
    private Duration timeout;

    // getters / setters
}
```

1.3. Регистрация автоконфигурации

Spring Boot 3+:

META-

```
INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports
```

Содержимое:

```
com.example.MyAutoConfiguration
```

⚠ Это обязательный шаг — без него автоконфигурация **не будет найдена**.

2 Модуль *-starter

Пустой модуль-заглушка, который:

- содержит **только зависимости**
- подтягивает *-autoconfigure
- предназначен для подключения пользователем

```
dependencies {  
    api project(":my-starter-autoconfigure")  
}
```

👉 Именно этот модуль пользователь пишет в build.gradle.

Как starter используется в приложении

```
implementation 'com.example:my-starter:1.0.0'
```

И всё.

Никаких @Enable, @Configuration и ручных бинов.

2. Как работает автоконфигурация Spring Boot (механизм внутри)?

Это **ключевой вопрос**, здесь важно показать понимание механики.

Общий процесс

1. Запускается `SpringApplication.run()`
 2. Создаётся `ApplicationContext`
 3. Spring Boot:
 - находит все автоконфигурации
 - фильтрует их по условиям
 - применяет подходящие
-

1 Поиск автоконфигураций

Spring Boot сканирует:

META-INF/spring/org.springframework.boot.autoconfigure.AutoConfiguration.imports

Из:

- `spring-boot-autoconfigure`
 - сторонних starter'ов
 - ваших кастомных starter'ов
-

2 Фильтрация через @Conditional

Каждая автоконфигурация включается **только если условия выполнены**.

Основные условия:

Аннотация	Назначение
@ConditionalOnClass	есть ли класс в classpath
@ConditionalOnMissingBean	нет пользовательского бина
@ConditionalOnProperty	включено ли свойство
@ConditionalOnBean	есть ли другой бин
@ConditionalOnWebApplication	web / reactive

👉 Это и есть “магия” Spring Boot.

3 Переопределение пользователем

```
@Bean
public MyClient myClient() {
    return new MyClient("custom");
}
```

- ➔ @ConditionalOnMissingBean **НЕ** выполнится
 - ➔ автоконфигурация **отключится автоматически**
-

4 Порядок автоконфигурации

- управляется через:
 - @AutoConfigureBefore
 - @AutoConfigureAfter
 - важен при сложных интеграциях (security, web, data)
-

Как это объяснить коротко

Spring Boot подключает автоконфигурации из classpath и применяет их только при выполнении условий, позволяя пользователю легко переопределять стандартное поведение.

3. Почему можно не указывать версии зависимостей?

Короткий ответ

Потому что Spring Boot использует **dependency management**.

Под капотом

Spring Boot:

- подключает **BOM (Bill Of Materials)**
 - для Maven:
 - spring-boot-dependencies
 - для Gradle:
 - dependency-management-plugin
-

Что делает BOM

- фиксирует **совместимые версии**:
 - Spring
 - Hibernate
 - Jackson
 - Logback
 - Tomcat
 - защищает от dependency hell
-

Пример

```
implementation 'com.fasterxml.jackson.core:jackson-databind'
```

Без версии — потому что:

- версия уже определена в `spring-boot-dependencies`
-

Когда версии указывать НУЖНО

- библиотека **не входит** в BOM
- нужна **другая версия**, чем в Boot
- подключаешь нестандартную интеграцию

```
implementation 'org.mapstruct:mapstruct:1.5.5.Final'
```

Важная фраза для собеседования

Spring Boot централизованно управляет версиями зависимостей через BOM, что обеспечивает совместимость библиотек и упрощает поддержку проекта.

Итог (коротко и мощно)

- Starter = зависимости + автоконфигурация
 - Автоконфигурация работает через `classpath` + `@Conditional`
 - Версии не указываются из-за BOM Spring Boot
-

1. Что такое Spring Security и для чего он используется?
 2. Как включить Spring Security в вашем Spring Boot приложении?
 3. Какие основные компоненты архитектуры Spring Security вы знаете?
-

1. Что такое Spring Security и для чего он используется?

Spring Security — это фреймворк для аутентификации, авторизации и защиты приложений на уровне:

- HTTP (web, REST)
- методов (service layer)
- интеграций (JWT, OAuth2, LDAP, SSO)

Какие задачи решает Spring Security

- аутентификация (кто ты?)
- авторизация (что тебе можно?)
- защита от атак:
 - CSRF
 - XSS
 - Session Fixation
 - Clickjacking
- управление сессиями
- интеграция с OAuth2 / JWT / LDAP

👉 **Важно:** Spring Security — это не только логин-форма, а полноценный security-фреймворк.

Коротко для собеседования

Spring Security используется для централизованного управления безопасностью приложения: аутентификацией, авторизацией и защитой от распространённых атак.

2. Как включить Spring Security в Spring Boot приложении?

Шаг 1 Подключить зависимость

```
implementation 'org.springframework.boot:spring-boot-starter-security'
```

➡ Этого уже достаточно, чтобы Security включился.

Что произойдёт автоматически

- все эндпоинты будут защищены
- появится default login page
- создастся пользователь user
- пароль выведется в лог при старте

👉 Это частый вопрос-ловушка.

Шаг 2 Настроить Security Configuration

В Spring Boot 3 / Spring Security 6:

```
@Configuration
@EnableWebSecurity
public class SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws
Exception {
        http
            .csrf(csrf -> csrf.disable())
            .authorizeHttpRequests(auth -> auth
                .requestMatchers("/public/**").permitAll()
                .anyRequest().authenticated()
            )
            .httpBasic();

        return http.build();
    }
}
```

⚠ WebSecurityConfigurerAdapter удалён.

Шаг 3 (опционально) Пользователь

```
@Bean
public UserDetailsService users() {
    return new InMemoryUserDetailsManager(
        User.withUsername("admin")
            .password("{noop}1234")
            .roles("ADMIN")
            .build()
    );
}
```

3. Основные компоненты архитектуры Spring Security

Здесь важно показать **понимание цепочки**, а не просто перечисление.

Архитектура Spring Security (high-level)

```
HTTP Request
↓
Security Filter Chain
↓
Authentication
↓
Authorization
↓
Controller
```

1 Security Filter Chain

- набор **Servlet Filters**
- перехватывает **каждый HTTP-запрос**

Примеры фильтров:

- UsernamePasswordAuthenticationFilter
 - BasicAuthenticationFilter
 - BearerTokenAuthenticationFilter
-

2 Authentication

Процесс проверки личности.

Ключевые интерфейсы:

- Authentication
- AuthenticationManager
- AuthenticationProvider

Пример:

- логин + пароль
 - JWT токен
 - OAuth2 access token
-

3 AuthenticationManager

- принимает Authentication
- делегирует провайдерам
- возвращает **аутентифицированный объект**

```
Authentication authenticate(Authentication authentication);
```

4 AuthenticationProvider

- реальная логика аутентификации
- может быть несколько провайдеров

Примеры:

- DaoAuthenticationProvider
 - JWT provider
 - LDAP provider
-

5 UserDetailsService

- загружает пользователя
- возвращает UserDetails

```
UserDetails loadUserByUsername(String username);
```

Источник:

- БД
 - LDAP
 - InMemory
-

6 SecurityContext

- хранит текущего пользователя
- доступен через:

```
SecurityContextHolder.getContext().getAuthentication();
```

7 Authorization

Проверка прав.

Реализуется через:

- `hasRole("ADMIN")`
- `hasAuthority("SCOPE_read")`
- `@PreAuthorize`

```
@PreAuthorize("hasRole('ADMIN')")
public void deleteUser() {}
```

Ключевая фраза для интервью

Spring Security построен вокруг цепочки фильтров, где сначала происходит аутентификация пользователя, а затем авторизация доступа к ресурсам.

Частые вопросы-ловушки

-  *Работает ли Security без конфигурации?*
 Да, с дефолтными настройками
 -  *Где хранится текущий пользователь?*
 В `SecurityContext`
 -  *Можно ли переопределить поведение?*
 Да, через `SecurityFilterChain` и бины
-

Короткий ответ (если мало времени)

Spring Security — это фреймворк для аутентификации и авторизации. В Spring Boot он включается добавлением starter'а и настраивается через `SecurityFilterChain`.
Архитектура построена на цепочке фильтров, `AuthenticationManager`, `AuthenticationProvider` и `SecurityContext`.

1. Как настроить аутентификацию пользователя с использованием базы данных в Spring Security?
 2. Как реализовать JWT аутентификацию в Spring Security?
 3. Как можно ограничить доступ к определенным URL в вашем приложении с использованием Spring Security?
-

1. Аутентификация через БД в Spring Security

Общая схема

```
HTTP request
→ Security Filter Chain
→ AuthenticationManager
→ DaoAuthenticationProvider
→ UserDetailsService
→ Database
```

Шаг 1 Сущность пользователя

```
@Entity
@Table(name = "users")
public class User {

    @Id @GeneratedValue
    private Long id;

    @Column(unique = true)
    private String username;

    private String password;

    @ElementCollection(fetch = FetchType.EAGER)
    private Set<String> roles;
}
```

Шаг 2 Repository

```
public interface UserRepository
    extends JpaRepository<User, Long> {

    Optional<User> findByUsername(String username);
}
```

Шаг 3 UserDetailsService

```
@Service
public class DbUserDetailsService implements UserDetailsService {

    private final UserRepository repository;

    public DbUserDetailsService(UserRepository repository) {
        this.repository = repository;
    }

    @Override
    public UserDetails loadUserByUsername(String username) {
        User user = repository.findByUsername(username)
            .orElseThrow(() -> new UsernameNotFoundException(username));

        return org.springframework.security.core.userdetails.User
            .withUsername(user.getUsername())
            .password(user.getPassword())
            .authorities(user.getRoles())
            .build();
    }
}
```

Шаг 4 PasswordEncoder

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();
}
```

 **Никогда не хранить пароли в открытом виде.**

Шаг 5 Security Config

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
            .anyRequest().authenticated()
        )
        .httpBasic();

    return http.build();
}
```

Ключевая фраза

Аутентификация через БД в Spring Security реализуется через `UserDetailsService`, который загружает пользователя из базы данных, и `DaoAuthenticationProvider`, который проверяет пароль с помощью `PasswordEncoder`.

2. JWT-аутентификация в Spring Security

Общая идея

- **stateless**
- без сессий
- клиент хранит токен
- сервер проверяет подпись токена

Шаг 1 Зависимость

```
implementation 'org.springframework.boot:spring-boot-starter-security'  
implementation 'io.jsonwebtoken:jjwt-api:0.11.5'
```

Шаг 2 Генерация JWT

```
public String generateToken(UserDetails user) {  
    return Jwts.builder()  
        .setSubject(user.getUsername())  
        .claim("roles", user.getAuthorities())  
        .setIssuedAt(new Date())  
        .setExpiration(new Date(System.currentTimeMillis() + 3600000))  
        .signWith(secretKey)  
        .compact()  
}
```

Шаг 3 JWT Filter

```
@Component  
public class JwtAuthenticationFilter extends OncePerRequestFilter {  
  
    @Override  
    protected void doFilterInternal(  
        HttpServletRequest request,  
        HttpServletResponse response,  
        FilterChain filterChain  
    ) throws ServletException, IOException {  
  
        String header = request.getHeader("Authorization");  
  
        if (header != null && header.startsWith("Bearer ")) {  
            String token = header.substring(7);  
            String username = jwtService.extractUsername(token);  
  
            Authentication auth =  
                new UsernamePasswordAuthenticationToken(  
                    username, null, jwtService.extractAuthorities(token)  
                );  
  
            SecurityContextHolder.getContext().setAuthentication(auth);  
        }  
  
        filterChain.doFilter(request, response);  
    }  
}
```


Шаг 4 Stateless Security Config

```
http
    .csrf(csrf -> csrf.disable())
    .sessionManagement(sm ->
        sm.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
    )
    .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/auth/**").permitAll()
        .anyRequest().authenticated()
    );
```

Ключевая фраза

JWT-аутентификация в Spring Security строится на stateless-филт্রে, который извлекает токен из запроса, валидирует его и устанавливает Authentication в SecurityContext.

3. Ограничение доступа к URL

1 Через HttpSecurity

```
.authorizeHttpRequests(auth -> auth
    .requestMatchers("/admin/**").hasRole("ADMIN")
    .requestMatchers("/user/**").hasAnyRole("USER", "ADMIN")
    .requestMatchers("/public/**").permitAll()
    .anyRequest().authenticated()
)
```

2 Через Method Security

```
@EnableMethodSecurity
@PreAuthorize("hasRole('ADMIN')")
public void deleteUser() {}
```

3 Через Authorities / Scopes (JWT)

```
.requestMatchers("/api/**").hasAuthority("SCOPE_read")
```

Важные моменты

- URL Security → coarse-grained
- Method Security → fine-grained
- В реальных проектах используют оба

1. Как настроить многомерную аутентификацию (Multi-Factor Authentication, MFA) с использованием Spring Security?
 2. Как использовать Spring Security для защиты REST API с помощью OAuth2?
 3. Как настроить собственный фильтр безопасности в Spring Security и в каких случаях это может быть необходимо?
-

1. Как настроить MFA (Multi-Factor Authentication) в Spring Security?

⚠ Важно сразу сказать на интервью:
Spring Security не предоставляет MFA “из коробки” — MFA реализуется комбинацией механизмов.

Типовая архитектура MFA

1. Username + Password
 2. One-Time Password (OTP) / TOTP / SMS / Email
 3. Полная аутентификация
-

Подход №1 — MFA как второй шаг после логина (рекомендуемый)

Шаг 1 Обычная аутентификация

```
UsernamePasswordAuthenticationToken
```

Шаг 2 Частично аутентифицированный пользователь

```
Authentication auth =  
    new UsernamePasswordAuthenticationToken(  
        userDetails,  
        null,  
        List.of(new SimpleGrantedAuthority("MFA_REQUIRED"))  
    );
```

➡ пользователь не полностью аутентифицирован

Шаг 3 Проверка OTP

- ввод TOTP / SMS / email code
- проверка через сервис (Google Authenticator, Authy)

```
if (otpService.verify(code, secret)) {  
    Authentication fullAuth =  
        new UsernamePasswordAuthenticationToken(  
            userDetails,  
            null,  
            userDetails.getAuthorities()  
        );  
  
    SecurityContextHolder.getContext()  
        .setAuthentication(fullAuth);  
}
```

Ограничение доступа

```
.authorizeHttpRequests(auth -> auth  
    .requestMatchers("/mfa/**").hasAuthority("MFA_REQUIRED")  
    .anyRequest().authenticated()  
)
```

Подход №2 — отдельный `AuthenticationProvider` для MFA

- кастомный `MfaAuthenticationToken`
 - отдельный `AuthenticationProvider`
 - сложнее, но чище архитектурно
-

Ключевая фраза для интервью

MFA в Spring Security реализуется как многошаговая аутентификация с использованием кастомных `Authentication` и дополнительной валидации после первичной проверки логина и пароля.

2. Защита REST API с OAuth2 в Spring Security

Базовая идея

- приложение — **Resource Server**
 - проверяет **JWT access token**
 - токен выдан **Authorization Server**
-

Шаг 1 Зависимости

```
implementation 'org.springframework.boot:spring-boot-starter-oauth2-resource-server'
```

Шаг 2 Настройка `application.yml`

```
spring:
  security:
    oauth2:
      resourceserver:
        jwt:
          issuer-uri: https://auth.example.com/realms/app
```

Spring автоматически:

- скачает JWK
 - настроит JWT decoder
 - включит фильтр
-

Шаг 3 Security Config

```
@Bean
SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.disable())
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/public/**").permitAll()
            .requestMatchers("/admin/**").hasRole("ADMIN")
            .anyRequest().authenticated()
        )
        .oauth2ResourceServer(oauth2 -> oauth2.jwt());

    return http.build();
}
```

Шаг 4 Использование claims

```
@PreAuthorize("hasAuthority('SCOPE_read')")
@GetMapping("/data")
public List<Data> data() {
    return service.get();
}
```

Важно для интервью

- REST API → **stateless**
 - без сессий
 - без CSRF
-

Ключевая фраза

Spring Security в режиме OAuth2 Resource Server валидирует JWT токены, проверяя подпись и claims, и использует их для авторизации запросов.

3. Собственный фильтр безопасности в Spring Security

Когда нужен кастомный фильтр

- JWT без OAuth2
 - API key authentication
 - MFA шаг
 - аудит / логирование
 - custom headers
-

Как написать фильтр

1 Наследоваться от `OncePerRequestFilter`

```
@Component
public class ApiKeyFilter extends OncePerRequestFilter {

    @Override
    protected void doFilterInternal(
        HttpServletRequest request,
        HttpServletResponse response,
        FilterChain filterChain
    ) throws ServletException, IOException {

        String apiKey = request.getHeader("X-API-KEY");

        if (isValid(apiKey)) {
            Authentication auth =
                new UsernamePasswordAuthenticationToken(
                    "api-client", null,
                    List.of(new SimpleGrantedAuthority("API"))
                );

            SecurityContextHolder.getContext()
                .setAuthentication(auth);
        }

        filterChain.doFilter(request, response);
    }
}
```

2 Подключить в `SecurityFilterChain`

```
http.addFilterBefore(
    apiKeyFilter,
    UsernamePasswordAuthenticationFilter.class
);
```

Важно понимать порядок фильтров

- до аутентификации → `addFilterBefore`
- после → `addFilterAfter`

Частые ошибки

- ❌ забыли `OncePerRequestFilter`
- ❌ не вызвали `filterChain.doFilter`
- ❌ кладут бизнес-логику в фильтр
- ❌ нарушают `stateless`

1. Что такое Spring Cache и зачем он используется?
 2. Какие аннотации предоставляет Spring Cache из коробки?
 3. В каких случаях имеет смысл использовать кэширование в приложении?
-

1. Что такое Spring Cache и зачем он используется?

Spring Cache — это абстракция Spring для кэширования результатов выполнения методов.

Он позволяет:

- ускорить работу приложения
- снизить нагрузку на БД и внешние сервисы
- централизованно управлять кэшем
- менять реализацию кэша **без изменения бизнес-кода**

👉 Важный момент:

Spring Cache **не является кэшем сам по себе**, он — **обёртка (abstraction)** над:

- Caffeine
 - Ehcache
 - Redis
 - Hazelcast
 - Simple (in-memory)
-

Короткая формулировка для интервью

Spring Cache — это абстракция, позволяющая кэшировать результаты методов с помощью аннотаций, не привязываясь к конкретной реализации кэша.

2. Аннотации Spring Cache из коробки

Spring Cache работает через **AOP (прокси)** — это важно упомянуть.

Основные аннотации

1 @EnableCaching

Включает поддержку кэширования.

```
@Configuration
@EnableCaching
public class CacheConfig {
}
```

2 @Cacheable

Кэширует результат метода.

```
@Cacheable("users")
public User getUser(Long id) {
    return repository.findById(id).orElseThrow();
}
```

Что происходит:

- при первом вызове → метод выполняется
 - результат кладётся в кэш
 - при повторном вызове → метод **не выполняется**
-

3 @CachePut

Всегда выполняет метод, но обновляет кэш.

```
@CachePut(value = "users", key = "#user.id")
public User update(User user) {
    return repository.save(user);
}
```

4 @CacheEvict

Удаляет данные из кэша.

```
@CacheEvict(value = "users", key = "#id")
public void delete(Long id) {
    repository.deleteById(id);
}
```

Очистка всего кэша:

```
@CacheEvict(value = "users", allEntries = true)
```

5 @Caching

Комбинация нескольких операций.

```
@Caching(
    evict = {
        @CacheEvict("users"),
        @CacheEvict("profiles")
    }
)
```

6 Условия и ключи

```
@Cacheable(
    value = "users",
    key = "#id",
    unless = "#result == null",
    condition = "#id > 0"
)
```

3. Когда имеет смысл использовать кэширование?

Это **ключевой вопрос** — здесь часто заваливаются.

Подходит для кэширования

- часто читаемые данные
 - редко изменяемые данные
 - дорогие операции:
 - запросы к БД
 - REST / SOAP вызовы
 - сложные вычисления
 - справочники
 - конфигурации
-

НЕ подходит

- данные, которые часто меняются
 - операции с побочными эффектами
 - транзакционные write-операции
 - критичные к актуальности данные (балансы, платежи)
-

Типовые кейсы

- `getUserById`
 - `findAllCountries`
 - `getExchangeRates`
 - `loadPermissions`
-

Важные нюансы (для senior-ответа)

1 Прокси и self-invocation

`this.getUser(id);` // ❌ кэш НЕ сработает

Причина — AOP-прокси.

2 Кэш ≠ транзакция

- кэш может вернуть устаревшие данные
 - всегда думать об **инвалидации**
-

3 Распределённый кэш

- несколько экземпляров → нужен Redis / Hazelcast
 - `ConcurrentHashMap` не подойдёт
-

4 TTL и eviction

- контролировать время жизни
 - избегать бесконечного роста кэша
-

Короткий ответ (если интервьюер торопит)

Spring Cache — это абстракция для кэширования результатов методов с помощью аннотаций. Он используется для ускорения приложения и снижения нагрузки. Основные аннотации — `@Cacheable`, `@CachePut`, `@CacheEvict`.

1. Как включить и настроить Spring Cache в Spring Boot-приложении?
 2. Как работает аннотация @Cacheable, и как определить ключ кэша?
 3. Чем отличается @CachePut от @Cacheable и @CacheEvict?
-

1. Как включить и настроить Spring Cache в Spring Boot-приложении?

Шаг 1 Подключить зависимость

Минимум:

```
implementation 'org.springframework.boot:spring-boot-starter-cache'
```

Реализация кэша (пример — Caffeine):

```
implementation 'com.github.ben-manes.caffeine:caffeine'
```

(или Redis / Ehcache — логика та же)

Шаг 2 Включить кэширование

```
@Configuration
@EnableCaching
public class CacheConfig {
}
```

⚠ В Spring Boot это обязательно, иначе аннотации не работают.

Шаг 3 Настроить CacheManager

Пример: Caffeine

```
@Bean
public CacheManager cacheManager() {
    CaffeineCacheManager manager = new CaffeineCacheManager("users");
    manager.setCaffeine(
        Caffeine.newBuilder()
            .expireAfterWrite(10, TimeUnit.MINUTES)
            .maximumSize(10_000)
    );
    return manager;
}
```

Альтернатива: auto-configuration

```
spring:
  cache:
    type: caffeine
```

Spring Boot сам создаст `CacheManager`.

Проверка, что кэш включён

- в логах появится `CacheManager`
 - методы с `@Cacheable` перестанут выполняться повторно
-

2. Как работает `@Cacheable` и как определить ключ кэша?

Базовое поведение

```
@Cacheable("users")
public User getUser(Long id) {
    return repository.findById(id).orElseThrow();
}
```

Что происходит:

1. Формируется ключ (по аргументам метода)
 2. Проверяется кэш
 3. Если значение есть → метод **не вызывается**
 4. Если нет → метод выполняется, результат кладётся в кэш
-

Определение ключа кэша

По умолчанию

- ключ = все аргументы метода
 - используется `SimpleKey`
-

Явный ключ (SpEL)

```
@Cacheable(value = "users", key = "#id")
```

Другие примеры:

```
key = "#user.id"  
key = "#root.methodName"  
key = "#root.targetClass.name + #id"
```

Условия кэширования

```
@Cacheable(  
    value = "users",  
    condition = "#id > 0",  
    unless = "#result == null"  
)
```

- `condition` — до выполнения
- `unless` — после выполнения

⚠ Частый вопрос на интервью.

Важный нюанс

```
this.getUser(id); // ❌ @Cacheable не работает
```

Причина — AOP-прокси (self-invocation).

3. Отличия @CachePut, @Cacheable, @CacheEvict

Аннотация	Выполняет метод	Работает с кэшем	Когда использовать
@Cacheable	❌ (если есть кэш)	кладёт	read-операции
@CachePut	✅ всегда	обновляет	update
@CacheEvict	✅ / ❌	удаляет	delete / invalidate

@CachePut

```
@CachePut(value = "users", key = "#user.id")
public User update(User user) {
    return repository.save(user);
}
```

- метод **выполняется всегда**
- результат **обновляет кэш**

@CacheEvict

```
@CacheEvict(value = "users", key = "#id")
public void delete(Long id) {
    repository.deleteById(id);
}
```

Очистка всего кэша:

```
@CacheEvict(value = "users", allEntries = true)
```

Типовой паттерн

```
@Cacheable("users")
User get(Long id);
```

```
@CachePut(value = "users", key = "#user.id")
User update(User user);
```

```
@CacheEvict(value = "users", key = "#id")
void delete(Long id);
```

Короткий ответ для интервью

@Cacheable кэширует результат и может пропускать выполнение метода, @CachePut всегда выполняет метод и обновляет кэш, а @CacheEvict удаляет данные из кэша.

Частые ошибки

-  забыли @EnableCaching
 -  неправильный ключ
 -  кэширование write-методов
 -  локальный кэш в кластере
-

1. Как настроить использование внешних систем кэширования, таких как Redis или Caffeine, в Spring Cache?
 2. Какие существуют подходы к инвалидации кэша в распределённой среде?
 3. Какие есть проблемы многопоточности при использовании кэша, и как их избежать (например, cache stampede)?
 4. Как разделить данные между разными серверами?(consistent hashing)
 5. Что такое "прогревание кеша"?
-

1. Как настроить Redis и Caffeine в Spring Cache?

◆ Redis (распределённый кэш)

Когда использовать

- несколько экземпляров приложения
 - Kubernetes / cloud
 - строгая консистентность между нодами
-

Шаг 1 Зависимости

```
implementation 'org.springframework.boot:spring-boot-starter-cache'
implementation 'org.springframework.boot:spring-boot-starter-data-redis'
```

Шаг 2 Конфигурация

```
spring:
  cache:
    type: redis
  data:
    redis:
      host: localhost
      port: 6379
```

Шаг 3 CacheManager

```
@Bean
public RedisCacheManager cacheManager(RedisConnectionFactory factory) {
    RedisCacheConfiguration config =
        RedisCacheConfiguration.defaultCacheConfig()
            .entryTtl(Duration.ofMinutes(10))
            .disableCachingNullValues();

    return RedisCacheManager.builder(factory)
        .cacheDefaults(config)
        .build();
}
```

Особенности Redis

- ✓ распределённый
 - ✓ TTL
 - ✗ сетевые задержки
 - ✗ сериализация
-

◆ Caffeine (локальный in-memory)

Когда использовать

- один инстанс
 - ultra-low latency
 - high read / low write
-

Зависимости

```
implementation 'com.github.ben-manes.caffeine:caffeine'
```

Конфигурация

```
@Bean
public CacheManager cacheManager() {
    CaffeineCacheManager manager =
        new CaffeineCacheManager("users");

    manager.setCaffeine(
        Caffeine.newBuilder()
            .expireAfterWrite(5, TimeUnit.MINUTES)
            .maximumSize(100_000)
    );
    return manager;
}
```

Особенности Caffeine

- ✓ очень быстрый
 - ✗ не распределённый
 - ✗ каждый инстанс — свой кэш
-

2. Инвалидация кэша в распределённой среде

⚠ Самая сложная часть кэширования.

Подходы

1 TTL (Time To Live) — самый популярный

Данные устаревают автоматически

- ✓ просто
 - ✗ возможна устаревшая информация
-

2 Явная инвалидация (@CacheEvict)

```
@CacheEvict(value = "users", key = "#id")
```

- ✗ сложно масштабировать
 - ✗ требует дисциплины
-

3 Pub/Sub (Redis)

```
Service A → publish "invalidate:user:42"  
Service B → evict cache
```

- ✓ быстро
 - ✗ сложнее в реализации
-

4 Write-through / Write-behind

- запись идёт **через кэш**
- кэш и БД синхронизированы

Используется редко, но критично в high-load системах.

Ключевая фраза для интервью

В распределённых системах чаще всего используется TTL в комбинации с явной инвалидацией или pub/sub механизмами.

3. Проблемы многопоточности и cache stampede



Cache Stampede (dog-pile effect)

TTL истёк
1000 потоков → БД
БД упала

Способы решения

1 Locking (single flight)

1 поток грузит данные
остальные ждут

Caffeine:

```
@Cacheable(sync = true)
```

Redis:

- Redisson lock
 - SETNX
-

2 Early expiration

- обновлять кэш до истечения TTL
-

3 Randomized TTL

TTL = 5–7 минут

Предотвращает одновременное истечение.

4 Stale-while-revalidate

- отдаём старые данные
 - обновляем асинхронно
-

Другие проблемы

- race conditions
 - dirty reads
 - thundering herd
-

4. Как разделить данные между серверами (Consistent Hashing)?

Проблема обычного hashing

$\text{hash}(\text{key}) \% N$

→ при добавлении сервера всё ломается

Consistent Hashing

- серверы располагаются на хэш-кольце
 - ключ маппится на ближайший узел
 - при добавлении сервера:
 - перераспределяется $\sim 1/N$ данных
-

Используется в:

- Redis Cluster
 - Cassandra
 - Kafka
 - Memcached
-

Виртуальные ноды

```
server1#1  
server1#2  
server1#3
```

👉 равномерное распределение нагрузки

Ключевая фраза

Consistent hashing минимизирует перераспределение данных при изменении количества серверов.

5. Что такое «прогревание кэша» (Cache Warm-up)?

Определение

Прогревание кэша — это предварительное заполнение кэша данными до начала реальной нагрузки.

Зачем нужно

- избежать cold start
 - предотвратить cache stampede
 - обеспечить стабильный latency после деплоя
-

Как реализуют

1 При старте приложения

```
@PostConstruct
public void warmUp() {
    service.getTopUsers();
}
```

2 Фоновыми job'ами

- cron
 - scheduler
-

3 Blue-Green deployment

- прогрев перед переключением трафика
-

Минусы

- ✗ увеличивает время старта
 - ✗ может нагрузить БД
-

Итог (коротко и сильно)

- Redis → distributed
- Caffeine → local & fast
- TTL + Pub/Sub → основа инвалидации
- Cache stampede → locking, jitter, stale data
- Consistent hashing → масштабирование
- Cache warm-up → стабильность после старта

1. Что такое Spring Cloud и для чего он используется?
 2. Какие основные проблемы решает Spring Cloud?
 3. Перечислите несколько проектов, входящих в состав Spring Cloud.
-

1. Что такое Spring Cloud и для чего он используется?

Spring Cloud — это набор инструментов и библиотек для **создания и управления распределёнными (микросервисными) системами** на базе Spring Boot.

Он предоставляет **готовые решения типовых проблем микросервисов**, позволяя:

- быстрее разрабатывать микросервисы
- снизить количество инфраструктурного кода
- стандартизировать подходы



Ключевая идея:

Spring Cloud **не заменяет инфраструктуру**, а **интегрируется с ней** (Service Discovery, Config Server, Gateway и т.д.).

Короткий ответ для интервью

Spring Cloud — это экосистема проектов, которая упрощает разработку микросервисов, предоставляя готовые решения для конфигурации, обнаружения сервисов, маршрутизации, отказоустойчивости и мониторинга.

2. Какие основные проблемы решает Spring Cloud?

Это **главный вопрос** — важно говорить именно про *проблемы*, а не просто перечислять технологии.

1 Централизованная конфигурация

Проблема:

- конфигурация разбросана по сервисам
- сложность смены окружений

Решение:

- Spring Cloud Config Server
- конфигурация хранится централизованно (Git, Vault)

2 Service Discovery

Проблема:

- сервисы динамически создаются и исчезают
- нельзя хардкодить адреса

Решение:

- Eureka / Consul / Zookeeper
 - динамическое обнаружение сервисов
-

3 Балансировка нагрузки

Проблема:

- несколько экземпляров сервиса
- куда отправлять запрос?

Решение:

- Spring Cloud LoadBalancer
 - client-side load balancing
-

4 Отказоустойчивость

Проблема:

- сбой одного сервиса “роняют” другие

Решение:

- Resilience4j
 - circuit breaker, retry, bulkhead, rate limiter
-

5 API Gateway

Проблема:

- клиенту нужно знать все сервисы
- cross-cutting concerns (security, logging)

Решение:

- Spring Cloud Gateway
 - единая точка входа
-

6 Distributed Tracing

Проблема:

- сложно отследить запрос через цепочку сервисов

Решение:

- Micrometer Tracing (ex Sleuth)
 - Zipkin / Tempo
-

7 Безопасность

Проблема:

- аутентификация между сервисами

Решение:

- OAuth2, JWT
 - интеграция со Spring Security
-

3. Проекты Spring Cloud (что перечислить на интервью)

Достаточно 5–7, важнее понимать назначение.

Ключевые проекты

Проект	Назначение
Spring Cloud Config	централизованная конфигурация
Spring Cloud Gateway	API Gateway
Spring Cloud Netflix Eureka	service discovery
Spring Cloud LoadBalancer	балансировка
Resilience4j	отказоустойчивость
Spring Cloud OpenFeign	declarative REST clients
Micrometer Tracing	distributed tracing

Устаревшие / важное уточнение

- Hystrix ✗ (deprecated)
- Ribbon ✗ (заменён Spring Cloud LoadBalancer)
- Sleuth ✗ (вошёл в Micrometer)

⚠ Хорошо сказать это на интервью — плюс в карму.

Ключевая фраза для интервью

Spring Cloud предоставляет инфраструктурные паттерны микросервисной архитектуры: конфигурацию, discovery, gateway, отказоустойчивость и трассировку.

Итог (коротко)

- Spring Cloud = инструменты для микросервисов
- Решает инфраструктурные проблемы
- Основан на интеграции с cloud-native средой
- Состоит из набора независимых проектов

1. Как Spring Cloud упрощает работу с конфигурационными файлами в микросервисной архитектуре?
 2. Чем отличается Spring Cloud Netflix от Spring Cloud?
 3. Как в Spring Cloud реализована балансировка нагрузки?
-

1. Как Spring Cloud упрощает работу с конфигурационными файлами в микросервисной архитектуре?

Проблема без Spring Cloud

- Множество сервисов → множество `application.properties` / `application.yml`
 - Разные окружения (dev, test, prod)
 - При изменении конфигурации нужно перезапускать все сервисы вручную
 - Высокий риск ошибок и несоответствия версий
-

Решение — Spring Cloud Config

Централизованный сервер конфигурации

- Один Config Server, который хранит конфигурацию в Git / Vault / локальных файлах
 - Сервисы подтягивают настройки динамически через HTTP
 - Поддержка профилей (`application-dev.yml`, `application-prod.yml`)
-

Пример использования

Config Server

```
@SpringBootApplication
@EnableConfigServer
public class ConfigServerApplication {}
```

application.yml (Config Server)

```
server:
  port: 8888
spring:
  cloud:
    config:
      server:
        git:
          uri: https://github.com/example/config-repo
```

Микросервис

```
spring:
  application:
    name: user-service
  cloud:
    config:
      uri: http://localhost:8888
```

✓ Плюсы:

- Конфигурация централизована
- Поддержка профилей и переменных
- Можно менять конфигурацию без перезапуска через `spring.cloud.bus + messaging (refresh)`

2. Чем отличается Spring Cloud Netflix от Spring Cloud?

Spring Cloud Netflix — это набор интеграций с проектами Netflix OSS, включающий:

- **Eureka** — service discovery
- **Ribbon** — client-side load balancing (deprecated)
- **Hystrix** — circuit breaker (deprecated)
- **Zuul** — API Gateway (устаревший)

Spring Cloud — более широкий набор инструментов для микросервисов, включая:

- Config Server
- Gateway (Spring Cloud Gateway, вместо Zuul)
- LoadBalancer (замена Ribbon)
- Resilience4j (вместо Hystrix)
- OpenFeign
- Micrometer Tracing (Sleuth)

⚠ Ключевой момент на собеседовании: **Netflix OSS интеграции устарели, современные сервисы используют Spring Cloud LoadBalancer и Resilience4j.**

3. Как в Spring Cloud реализована балансировка нагрузки?

1 Client-side load balancing (обычно через Feign + LoadBalancer)

- Клиент сам выбирает инстанс сервиса
- Использует список доступных сервисов из Eureka / Consul
- Round-robin или другие стратегии
- Spring Cloud LoadBalancer заменяет Ribbon

```
@LoadBalanced
@Bean
RestTemplate restTemplate() {
    return new RestTemplate();
}

// Использование
restTemplate.getForObject("http://user-service/users/1", User.class);
```

`http://user-service` — виртуальное имя сервиса, LoadBalancer выбирает конкретный инстанс.

2 Server-side load balancing (API Gateway)

- Spring Cloud Gateway принимает все запросы
- Проксирует на конкретный инстанс
- Можно использовать фильтры, rate limiting, retries

```
spring:
  cloud:
    gateway:
      routes:
        - id: user-service
          uri: lb://user-service
          predicates:
            - Path=/users/**
```

`lb://` → автоматически подключается LoadBalancer для выбора инстанса.

Ключевые фразы для интервью

- **Client-side LB:** клиент выбирает инстанс (@LoadBalanced RestTemplate / Feign)
- **Server-side LB:** gateway распределяет запросы (Spring Cloud Gateway)
- Современные проекты используют **Spring Cloud LoadBalancer** вместо Ribbon.

1. Опишите процесс настройки централизованного логирования в Spring Cloud.
 2. Каким образом Spring Cloud Stream обрабатывает сообщения в микросервисной архитектуре?
 3. Как в Spring Cloud реализован паттерн цепочки обязанностей (Chain of Responsibility) для обработки запросов?
-

1. Централизованное логирование в Spring Cloud

Проблема

- Множество микросервисов → отдельные логи
 - Сложно отслеживать запрос через несколько сервисов
 - Трудно анализировать ошибки в production
-

Решение: ELK / EFK Stack + Spring Cloud

1. **Логирование из приложения**
 - Используем Logback или Log4j2
 - Добавляем **корреляционный идентификатор** (traceId) через Spring Cloud Sleuth / Micrometer Tracing
 2. @Bean
 3.

```
public Sampler defaultSampler() {
```
 4.

```
    return Sampler.ALWAYS_SAMPLE;
```
 5.

```
}
```

 - Sleuth добавляет traceId и spanId в MDC (Mapped Diagnostic Context)
 6. **Сбор логов**
 - Filebeat / Fluentd / Logstash читают локальные файлы логов и отправляют в ELK/EFK
 - Можно использовать **Spring Cloud Logstash** напрямую
 7. **Хранение и визуализация**
 - Elasticsearch хранит логи
 - Kibana предоставляет дашборды и поиск
 - Grafana (опционально) для метрик и алертов
-

Пример конфигурации Logback + Logstash

```
<appender name="LOGSTASH"
class="net.logstash.logback.appender.LogstashTcpSocketAppender">
  <destination>localhost:5000</destination>
  <encoder class="net.logstash.logback.encoder.LogstashEncoder"/>
</appender>

<root level="INFO">
  <appender-ref ref="LOGSTASH"/>
</root>
```

✓ Ключевой момент: **корреляционный traceId** позволяет связать логи между сервисами.

2. Spring Cloud Stream и обработка сообщений

Что такое Spring Cloud Stream

- Абстракция над брокерами сообщений (Kafka, RabbitMQ)
 - Позволяет писать **реактивные / event-driven микросервисы** без привязки к конкретной технологии
-

Основные концепции

- **Bindings** — связывают канал с брокером
 - **Processor / Sink / Source** — функциональные интерфейсы
 - **@EnableBinding / functional programming model** — создаём обработчики
-

Пример: слушатель сообщений

```
@Bean
public Consumer<Message<String>> process() {
    return message -> {
        System.out.println("Received: " + message.getPayload());
        // бизнес-логика
    };
}
```

application.yml

```
spring:
  cloud:
    stream:
      bindings:
        process-in-0:
          destination: user-events
          group: user-service
      function:
        definition: process
```

✓ Особенности:

- Сообщения могут быть **многопоточными**
- Поддержка **групп / consumer groups**
- Полезно для **event-driven микросервисов**, реактивной обработки и decoupling

3. Chain of Responsibility (Цепочка обязанностей) в Spring Cloud

Контекст

- Паттерн CoR позволяет обработать запрос через последовательность обработчиков
- Часто применяется в:
 - API Gateway
 - Spring Cloud Gateway Filters
 - Security Filters

Spring Cloud Gateway пример

```
@Bean
public RouteLocator routes(RouteLocatorBuilder builder) {
    return builder.routes()
        .route("user_route", r -> r.path("/users/**")
            .filters(f -> f
                .addRequestHeader("X-Request-ID",
                    UUID.randomUUID().toString())
                .retry(3)
                .circuitBreaker(c ->
                    c.setName("userCB").setFallbackUri("/fallback"))
            )
            .uri("lb://user-service"))
        .build();
}
```

- Каждый **фильтр** → отдельный обработчик
- Filters выполняются в цепочке:
 1. Retry
 2. CircuitBreaker
 3. Logging / tracing
 4. Request modification

Security Filters Chain

- Spring Security реализует CoR через **SecurityFilterChain**
- Каждый фильтр может:
 - Пропустить запрос
 - Заблокировать
 - Добавить атрибуты в контекст

```
http
    .addFilterBefore(jwtFilter, UsernamePasswordAuthenticationFilter.class)
    .authorizeHttpRequests(auth -> auth.anyRequest().authenticated());
```

✅ Это пример **Chain of Responsibility** в микросервисной архитектуре.

Итог по трём вопросам

Тема	Ключевой момент
Централизованное логирование	ELK/EFK + traceId через Sleuth/Micrometer
Spring Cloud Stream	Абстракция над брокером, binding + consumer/producer, event-driven
Chain of Responsibility	Gateway Filters / SecurityFilterChain, последовательная обработка запросов